~\Downloads\python_generators_and_iterators_practice_set.md

Python Iterators and Generators: 20 Practice Questions

A progressive problem set covering the iterator protocol (`__iter__`, `__next__`), generator functions (`yield`, `yield from`), generator expressions, infinite iterators, memory efficiency, and coroutine-like methods (`send()`, `throw()`, `close()`).

Part 1: Questions

1. Basic Iterator Protocol: Custom Range

Implement a custom class named `ReverseRange` that implements Python's iterator protocol without using built-in `reversed()` or generators. It must take a start and stop integer and iterate backwards from `start` down to `stop` (inclusive).

2. Generator Function: Even Fibonacci Numbers

Write a generator function `even_fibonacci(limit)` that yields Fibonacci numbers that are even and strictly less than `limit`.

3. Generator Expressions: Memory-Efficient File Reader

Write a one-line generator expression (not a function) assigned to a variable `clean_lines` that takes an open file object `f`, strips all leading/trailing whitespace, and ignores lines that start with `#` or are completely empty.

4. Reusing Iterators vs. Generators

Explain what happens when you execute the following snippet. What will be printed by the second loop, and why?

```
gen = (x * x for x in range(3))
for a in gen:
    print(a, end=" ")
for b in gen:
```

```
    print(b, end=" ")
```

5. Infinite Generator with Step

Create a generator function `cycle_slice(iterable, n)` that indefinitely cycles through an iterable, yielding slices of size `n` as tuples. If the remaining elements cannot form a full tuple of size `n`, wrap around to the beginning to complete it.

6. Flattening Arbitrarily Nested Structures

Using `yield from` and recursion, write a generator `deep_flatten(nested_list)` that yields all scalar values from a structure with arbitrary levels of nested lists, tuples, or sets, while leaving strings intact as single scalar units.

7. The `iter()` Sentinel Variant

Python's `iter()` function accepts two arguments: `iter(callable, sentinel)`. Write a function `read_chunks(file_path, chunk_size=1024)` that uses this two-argument form of `iter()` to read a binary file in chunks until an empty byte string `b""` is encountered.

8. Custom State Machine Iterator

Implement an iterator class `TrafficLight` that cycles through `"RED" -> "GREEN" -> "YELLOW" -> "RED"` indefinitely. It must maintain an internal state and raise no exceptions.

9. Chaining Generators for ETL Pipelines

Create a data processing pipeline using three chained generator functions:

1. `parse_integers(raw_data)`: Takes a list of strings and yields only valid integers.
2. `filter_primes(numbers)`: Takes an iterable of integers and yields only prime numbers.
3. `squared_stream(numbers)`: Takes an iterable of numbers and yields their squares.

10. Memory Comparison: List vs. Generator

Write a Python script using `sys.getsizeof` to compare the memory consumed by a list containing the first 10^6 squares versus a generator expression computing the same values. Explain the output.

11. Generator `send()` Method

Write a generator `running_average()` that uses `yield` as an expression with `.send(val)`. It should receive numbers dynamically, compute the cumulative moving average, and yield the updated average on each step.

12. Handling Exceptions in Generators: `throw()`

Write a generator `resilient_counter()` that yields sequential integers starting from 0. If a custom exception `SkipStep` is thrown into the generator using `.throw()`, it should skip the current number by incrementing by 2 instead of 1, without terminating.

13. Resource Cleanup with `try...finally` and `close()`

Write a generator function `managed_stream(filename)` that opens a file, yields lines stripped of newline characters, and ensures the file is reliably closed even if the consumer calls `.close()` early or an unhandled exception occurs downstream.

14. Implementing `itertools.accumulate`

Implement your own version of `accumulate(iterable, func=operator.add)` as a generator without importing `itertools`. It should yield running reductions of items from the input iterable.

15. The `StopIteration` Return Value

Prior to PEP 380 / PEP 479, `return value` inside a generator had specific behavior. Explain how Python currently treats a `return "finished"` statement inside a generator function, and show how a consumer can retrieve this returned value.

16. Windowing Generator: Pairwise

Write a generator `pairwise(iterable)` that yields successive overlapping pairs as tuples: `(s0, s1), (s1, s2), (s2, s3), ...`. If the iterable has fewer than two elements, it should yield nothing.

17. Batching / Chunking an Iterator

Implement a generator `batched(iterable, batch_size)` that accepts any arbitrary iterator or iterable and yields items in non-overlapping lists of length `batch_size`. The final batch may be smaller if the total length is not divisible by `batch_size`.

18. Lookahead / Peekable Iterator

Design a class `Peekable` that wraps any iterator and provides a `.peek()` method allowing the user to view the next element without advancing the iterator, alongside normal iteration behavior.

19. Two-way Generator Pipeline with `yield from`

Demonstrate generator delegation: Write a sub-generator `subgen()` that accepts values via `yield` and returns a total sum when a negative number is sent. Write a delegating generator `delegator()` using `yield from subgen()` that captures the return value.

20. Tree Traversal using Recursive Generators

Given a binary tree node defined as:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Write an in-order traversal generator `inorder(root)` that yields values using `yield from` with minimal auxiliary space.